

甲方配置、Seed、数据导入、测试与部署规范

适用项目：Plush Toy ERP

适用阶段：客户试用候选 / 私有化部署候选

目标：规范每个甲方的配置包、初始化 seed、历史数据导入、测试验证、部署发布、回滚和签收流程。

核心原则：每个甲方都是一个独立交付单元；配置、seed、导入、部署必须可审计、可复现、可回滚、可测试。

1. 为什么需要这份规范

ERP 私有化项目和普通 SaaS 不同。每个甲方通常都有自己的：

- 公司名称和品牌。
- 菜单开关。
- 角色和权限。
- 字段叫法。
- 单据编号规则。
- 默认仓库。
- 默认单位。
- 物料分类。
- 产品分类。
- 客户/供应商历史数据。
- SKU / BOM / 库存期初数据。
- 历史订单。
- 财务期初或业务应收应付。
- 导入文件格式。
- 部署环境。
- 账号体系。
- 数据保密要求。

如果没有统一规范，很容易出现：

- A 客户配置污染 B 客户。
- demo seed 进入生产。
- 导入数据重复。
- 历史文件丢失或无法复现。
- 期初库存和库存流水不一致。
- 客户字段含义映射错误。
- 生产部署无法回滚。
- 上线后无法证明导入过什么、跳过了什么、失败了什么。

所以必须把每个甲方作为一个可管理的交付包。

2. 核心原则

2.1 每个甲方一个独立配置包

每个甲方必须有独立目录：

```
customers/  
  <customer-code>/  
    config/  
    seed/  
    import/  
    deployment/  
    evidence/  
    reports/
```

示例：

```
customers/  
  yoyoosun/  
    config/  
    seed/  
    import/  
    deployment/  
    evidence/  
    reports/
```

注意：

- 真实客户原始文件不建议直接提交到产品源码仓库。
- 可以提交脱敏样例、manifest、字段映射、导入报告。
- 真实文件应放在受控存储，并通过 hash、manifest、签收记录关联。

2.2 配置、seed、导入必须分开

三者不能混在一起。

类型	作用	是否可重复执行	是否可删除
Config	控制系统行为、菜单、字段、品牌、开关	是	可替换
Seed	初始化必要主数据、角色、权限、默认值	必须幂等	不随意删除
Import	导入客户历史业务数据	需要批次控制	不可随意重跑
Demo Seed	演示数据	只能 dev/demo	生产禁止

类型	作用	是否可重复执行	是否可删除
Test Fixture	自动化测试数据	只能测试环境	生产禁止

2.3 生产环境禁止 demo/test 数据

生产或客户试用环境禁止：

```
demo customer
demo supplier
test order
test SKU
test password
mock SMS code
debug seed
debug cleanup
sample finance fact
fake shipment
```

生产或客户试用环境只允许：

```
system seed
customer config
customer-approved seed
customer import data
customer migration data
```

2.4 所有导入必须 dry-run 先行

任何客户数据导入都必须先：

```
manifest check
schema check
mapping check
duplicate check
referential integrity check
business invariant check
dry-run report
customer confirmation
backup
apply
post-apply verification
```

不允许直接 apply。

2.5 所有客户数据导入必须可追溯

每次导入必须记录：

```
import_job_id
customer_code
source_manifest_version
source_file_hash
mapping_version
operator
started_at
finished_at
dry_run_report
apply_report
unresolved_queue
created_count
updated_count
skipped_count
failed_count
rollback_plan
```

3. 客户目录结构规范

推荐结构：

```
customers/
  <customer-code>/
    README.md

  config/
    customer.config.json
    menu.config.json
    field-alias.config.json
    numbering.config.json
    feature-flags.config.json
    permissions.config.json
    deployment.env.example

  seed/
    system-seed.json
    roles-seed.json
    permissions-seed.json
    masterdata-seed.json
```

```
warehouse-seed.json
unit-seed.json
seed-manifest.json

import/
  source-manifest.json
  mapping/
    customer.mapping.json
    supplier.mapping.json
    contact.mapping.json
    product.mapping.json
    sku.mapping.json
    material.mapping.json
    bom.mapping.json
    inventory-opening.mapping.json
    sales-order.mapping.json
    finance-opening.mapping.json
  samples/
    sanitized-customer-sample.xlsx
    sanitized-bom-sample.xlsx
  unresolved/
    unresolved-queue.sample.json

deployment/
  docker-compose.override.example.yml
  migration-plan.md
  deployment-checklist.md
  rollback-plan.md
  backup-plan.md
  smoke-test-plan.md

reports/
  config-lint-report.json
  seed-dry-run-report.json
  import-dry-run-report.json
  import-apply-report.json
  post-apply-verification-report.json
  smoke-test-report.json
  backup-restore-report.json
  security-scan-report.json

evidence/
  customer-signoff.md
  deployment-evidence.md
  import-evidence.md
```

4. 客户配置规范

4.1 `customer.config.json`

作用

定义甲方基础信息和部署标识。

示例

```
{
  "version": 1,
  "customerCode": "yoyoosun",
  "customerName": "YoyooSun",
  "deploymentType": "private",
  "timezone": "Asia/Shanghai",
  "locale": "zh-CN",
  "defaultCurrency": "CNY",
  "defaultLanguage": "zh-CN",
  "industry": "plush-toy-manufacturing",
  "enabled": true
}
```

字段说明

字段	必填	说明
<code>version</code>	是	配置版本
<code>customerCode</code>	是	稳定客户编码，不可随意变更
<code>customerName</code>	是	客户显示名称
<code>deploymentType</code>	是	private / trial / demo
<code>timezone</code>	是	时间解释规则
<code>locale</code>	是	语言和格式
<code>defaultCurrency</code>	是	默认币种
<code>defaultLanguage</code>	是	默认语言
<code>industry</code>	是	行业模板
<code>enabled</code>	是	是否启用

注意事项

- `customerCode` 必须稳定，不能用中文名。
- 不要把客户真实 secret 放进该文件。
- 生产环境 secret 通过环境变量或 Secret Manager 注入。

- timezone 必须明确，否则历史日期导入会出错。

4.2 menu.config.json

作用

控制甲方启用哪些菜单。

示例

```
{
  "version": 1,
  "menus": {
    "dashboard": true,
    "tasks": true,
    "masterData.customers": true,
    "masterData.suppliers": true,
    "masterData.products": true,
    "masterData.skus": true,
    "masterData.materials": true,
    "masterData.bom": true,
    "sales.orders": true,
    "purchase.orders": true,
    "purchase.receipts": true,
    "quality.inspections": true,
    "inventory.balances": true,
    "inventory.txns": true,
    "inventory.reservations": true,
    "production.tasks": true,
    "outsourcing.tasks": true,
    "shipments": true,
    "finance.facts": true,
    "imports": true,
    "reports": false,
    "system.audit": true,
    "legacyArchive": false
  }
}
```

注意事项

- 菜单隐藏不是权限控制。
- 后端 API 必须仍然检查权限。
- 客户试用阶段不成熟模块可以隐藏。
- 隐藏模块不能被路由直接访问绕过。

4.3 field-alias.config.json

作用

适配甲方自己的字段叫法。

示例

```
{
  "version": 1,
  "aliases": {
    "customer": {
      "customerName": "客户名称",
      "customerOrderNo": "客户订单号"
    },
    "product": {
      "skuCode": "款号",
      "spec": "规格/颜色"
    },
    "material": {
      "materialCode": "物料编号",
      "materialName": "材料名称"
    }
  }
}
```

注意事项

- 字段别名只影响 UI 显示。
- 不应该改变数据库字段含义。
- 不允许同一页面两个字段显示成同一个名字。
- 导入映射应使用系统标准字段，不直接使用 UI alias。

4.4 numbering.config.json

作用

定义单据编号规则。

示例

```
{
  "version": 1,
  "rules": {
    "salesOrder": "SO-{yyyyMMdd}-{seq4}",
    "purchaseOrder": "PO-{yyyyMMdd}-{seq4}",
    "purchaseReceipt": "PR-{yyyyMMdd}-{seq4}",
  }
}
```



```

    "qualityInspection": "QC-{yyyyMMdd}-{seq4}",
    "inventoryTxn": "ITX-{yyyyMMdd}-{seq6}",
    "productionTask": "MO-{yyyyMMdd}-{seq4}",
    "outsourcingTask": "OS-{yyyyMMdd}-{seq4}",
    "shipment": "SH-{yyyyMMdd}-{seq4}",
    "financeFact": "FIN-{yyyyMMdd}-{seq4}"
  }
}

```

注意事项

- 编号规则必须保证唯一。
- 编号生成必须在数据库事务中防并发冲突。
- 导入历史单据时，要区分“客户原单号”和“系统单号”。
- 不要用客户原始 Excel 行号当系统单号。

4.5 feature-flags.config.json

作用

控制功能开关。

示例

```

{
  "version": 1,
  "features": {
    "publicRegister": false,
    "smsMock": false,
    "debugSeed": false,
    "debugCleanup": false,
    "sqlArgsTracing": false,
    "legacyArchive": true,
    "financeLedger": false,
    "barcodeScan": false,
    "attachmentUpload": false,
    "pdaMode": false,
    "customerImportApply": false
  }
}

```

生产禁止开启

```

publicRegister
smsMock
debugSeed

```

```
debugCleanup
sqlArgsTracing
```

注意事项

- feature flag 必须进入 production preflight。
- 客户试用前不要默认开启未完成能力。
- hidden feature 不能绕过权限和 API 校验。

4.6 permissions.config.json

作用

定义甲方角色和权限矩阵。

角色建议

```
super_admin
admin
sales
purchase
warehouse
quality
production
outsourcing
finance
viewer
```

示例

```
{
  "version": 1,
  "roles": [
    {
      "role": "sales",
      "permissions": [
        "customer.read",
        "salesOrder.read",
        "salesOrder.create",
        "salesOrder.updateDraft",
        "salesOrder.confirm"
      ]
    },
    {
      "role": "warehouse",
      "permissions": [
        "inventory.read",

```

```

        "inventory.reserve",
        "shipment.read",
        "shipment.ship"
    ]
}
]
}

```

注意事项

- 权限配置必须由后端强制。
- 前端菜单只是展示控制。
- 初始 super admin 只能 bootstrap 一次。
- 不允许 seed 把普通用户自动提升为 super admin。

5. Seed 数据规范

5.1 Seed 分类

Seed 类型	说明	是否生产允许
System Seed	系统必需状态、枚举、默认配置	是
Role Seed	角色	是
Permission Seed	权限	是
Masterdata Seed	客户确认过的基础主数据	是
Unit Seed	单位和换算	是
Warehouse Seed	默认仓库和库位	是
Demo Seed	演示订单、演示客户、演示库存	否
Test Fixture	自动化测试数据	否
Import Seed	从客户文件导入的初始业务数据	是，但必须有报告

5.2 Seed 必须幂等

Seed 脚本必须支持重复执行。

重复执行时：

不重复创建角色
 不重复创建权限
 不重复创建默认仓库

不重复创建单位
不重复创建 admin
不重复写库存期初
不重复创建业务单据

每条 seed 应该有稳定 key：

```
seed_key  
customer_code  
entity_type  
entity_code  
version
```

5.3 seed-manifest.json

作用

记录 seed 文件、版本、hash、执行顺序。

示例

```
{  
  "version": 1,  
  "customerCode": "yoyoosun",  
  "seeds": [  
    {  
      "seedId": "roles-v1",  
      "file": "roles-seed.json",  
      "type": "roles",  
      "sha256": "",  
      "required": true,  
      "order": 10  
    },  
    {  
      "seedId": "permissions-v1",  
      "file": "permissions-seed.json",  
      "type": "permissions",  
      "sha256": "",  
      "required": true,  
      "order": 20  
    },  
    {  
      "seedId": "warehouse-v1",  
      "file": "warehouse-seed.json",  
      "type": "warehouse",  
      "sha256": "",  
      "required": true,  
      "order": 30  
    }  
  ]  
}
```

```
    "order": 30
  }
]
}
```

验收标准

- seed 执行顺序明确。
- 每个 seed 有 hash。
- required seed 缺失必须失败。
- seed 重复执行结果一致。
- seed 报告记录 created / updated / skipped。

5.4 Seed 执行报告

每次 seed 必须输出：

```
{
  "customerCode": "yoyoosun",
  "seedBatchId": "seed-20260611-001",
  "status": "passed",
  "created": 10,
  "updated": 2,
  "skipped": 30,
  "failed": 0,
  "errors": [],
  "startedAt": "2026-06-11T10:00:00+08:00",
  "finishedAt": "2026-06-11T10:00:30+08:00"
}
```

5.5 Seed 注意事项

不能做

不能在生产 seed 中创建 demo 订单
不能创建默认弱密码
不能创建 mock 手机验证码
不能写真实 secret
不能跳过权限检查创建 super admin
不能重复写期初库存
不能把 test fixture 放入客户试用环境

必须做

```
seed 前备份数据库
seed 前检查 APP_ENV
seed 前执行 preflight
seed 后生成报告
seed 后做 smoke test
seed 后写 audit event
```

6. 客户数据导入规范

6.1 导入阶段

标准流程：

1. Data Intake：接收客户文件
2. Data Classification：分类文件
3. Source Manifest：生成源文件清单
4. Mapping：字段映射
5. Validation：基础校验
6. Dry-run：预导入
7. Unresolved Queue：处理未匹配
8. Customer Confirmation：客户确认
9. Backup：导入前备份
10. Apply：正式导入
11. Post-apply Verification：导入后验证
12. Signoff：签收

6.2 原始文件接收注意事项

必须记录

```
文件名
文件大小
sha256
文件类型
来源人
接收时间
是否原始文件
是否脱敏
是否客户确认
```

不建议

把客户 raw files 直接提交到产品仓库
在聊天记录中传递长期保存的敏感文件
重命名后丢失 originalName
只靠中文文件名定位文件
不记录 hash

6.3 source-manifest.json

作用

让导入工具不依赖中文文件名、系统路径或人工记忆。

示例

```
{
  "version": 1,
  "customerCode": "yoyoosun",
  "receivedAt": "2026-06-11T10:00:00+08:00",
  "sources": [
    {
      "sourceId": "customer-master-v1",
      "type": "customer_master",
      "originalName": "客户资料.xlsx",
      "storedName": "customer-master-v1.xlsx",
      "sha256": "",
      "required": true,
      "containsPII": true,
      "sanitized": false,
      "notes": ""
    },
    {
      "sourceId": "bom-26029-v1",
      "type": "bom",
      "originalName": "26029#夜樱烬色材料明细表2026-1-19.xlsx",
      "storedName": "bom-26029-v1.xlsx",
      "sha256": "",
      "required": true,
      "containsPII": false,
      "sanitized": false,
      "notes": ""
    }
  ]
}
```

必须校验

```
sourceId 唯一
type 合法
storedName 存在
sha256 匹配
required 文件不可缺失
文件大小合理
文件扩展名符合 type
中文 originalName 不影响导入
```

6.4 字段映射规范

每种导入类型都要有 mapping 文件。

示例：

```
import/mapping/customer.mapping.json
import/mapping/supplier.mapping.json
import/mapping/product.mapping.json
import/mapping/sku.mapping.json
import/mapping/material.mapping.json
import/mapping/bom.mapping.json
import/mapping/inventory-opening.mapping.json
import/mapping/sales-order.mapping.json
```

mapping 示例

```
{
  "version": 1,
  "sourceType": "customer_master",
  "targetEntity": "customers",
  "fields": [
    {
      "sourceColumn": "客户名称",
      "targetField": "customer_name",
      "required": true,
      "transform": "trim"
    },
    {
      "sourceColumn": "客户编号",
      "targetField": "customer_code",
      "required": false,
      "transform": "trim_upper"
    }
  ]
},
```



```
"dedupe": {  
  "keys": ["customer_code", "customer_name"],  
  "strategy": "match_or_create"  
}  
}
```

6.5 导入顺序

推荐顺序：

1. 单位 units
2. 仓库 warehouses / locations
3. 客户 customers
4. 供应商 suppliers
5. 联系人 contacts
6. 产品 products
7. SKU product_skus
8. 物料 materials
9. BOM headers / items
10. 期初库存 inventory opening
11. 历史销售订单 sales orders
12. 历史采购订单 purchase orders
13. 历史收货 / 质检 / 库存流水
14. 历史出货 shipments
15. 期初应收 / 应付 finance facts

原因：

后面的业务单据依赖前面的主数据。
BOM 依赖 SKU、物料、单位。
库存依赖 SKU/物料、仓库、批次。
销售订单依赖客户和 SKU。
采购订单依赖供应商和物料。
财务事实依赖销售/采购/出货/外协来源。

7. 关键导入注意事项

7.1 客户和供应商

注意

- 客户名称可能重复。
- 客户简称可能不是唯一。

- 客户编码可能缺失。
- 电话、联系人可能混在一个单元格。
- 客户和供应商可能同名。
- 旧系统里的客户状态可能不规范。

测试

重复客户识别
缺失客户编号处理
同名不同客户处理
客户联系人拆分
禁用客户不允许新订单

7.2 产品和 SKU

注意

- 产品和 SKU 不要混为一谈。
- 客户款号不一定等于系统 SKU。
- 同一产品可能有颜色、尺寸、包装版本。
- SKU code 必须稳定唯一。
- 旧资料可能只有产品名，没有 SKU。

测试

SKU code 唯一
客户 SKU 映射正确
同产品多个 SKU 正确关联
禁用 SKU 不可下单
SKU 可关联 BOM 和库存

7.3 物料

注意

- 物料名称可能不规范。
- 同一物料可能有不同颜色、规格。
- 单位可能不统一。
- 默认供应商可能不唯一。
- 安全库存可能缺失。

测试

物料去重
单位映射

默认供应商映射
物料类别映射
物料用于 BOM

7.4 单位和换算

注意

单位是导入高风险点。

常见问题：

个 / 只 / pcs 混用
箱和个未设置换算
米和码混用
kg 和 g 混用
客户 Excel 中单位为空

测试

未知单位进入 unresolved queue
单位换算正确
数量转换后精度正确
BOM 用量单位合法
库存单位和 SKU 默认单位兼容

7.5 BOM

注意

BOM 是最容易导入错的核心数据。

必须确认：

BOM 属于哪个产品/SKU
BOM version 是什么
BOM 是否生效
物料是否都能匹配
用量单位是否正确
损耗率是否正确
替代料如何表达
同 SKU 是否已有 active BOM

测试

BOM item qty > 0
loss_rate >= 0
所有物料必须匹配
所有单位必须匹配
同 SKU 只能有一个 ACTIVE BOM
ACTIVE BOM 不允许直接覆盖
导入新版本必须生成 DRAFT 或归档旧版本

7.6 期初库存

注意

期初库存不能随便写库存余额。

必须决定：

是否生成 OPENING_BALANCE 库存流水
期初日期是什么
仓库/库位是什么
批次是否需要
质量状态是什么
是否待检
是否可用

推荐做法

期初库存通过 inventory_txns 写入
txn_type = OPENING_BALANCE
source_type = CUSTOMER_IMPORT
source_id = import_job_id

禁止

直接 update inventory_balances 而没有流水
重复导入导致库存翻倍
没有仓库的库存
没有单位的库存
待检/不良库存计入可用库存

测试

期初库存导入后余额等于流水聚合
重复 apply 不重复增加库存
缺少仓库进入 unresolved queue
负库存进入错误
未知 SKU/物料进入 unresolved queue

7.7 历史销售订单

注意

历史订单可能有不同状态：

未确认
已确认
生产中
部分出货
已完成
已取消

必须映射到系统状态。

必须区分

客户原订单号
系统销售订单号
导入批次
历史状态
当前是否仍需执行

测试

客户订单号重复处理
客户不存在时阻塞
SKU 不存在时阻塞
已完成历史订单不触发生产/采购
未完成订单可进入后续流程
取消订单不生成库存预留

7.8 历史采购订单和收货

注意

采购订单、收货、库存三者必须一致。

导入历史采购时必须确认：

采购订单是否仍未收完
历史收货是否已入库
是否需要质检
是否已经付款

测试

收货数量不得超过采购数量
已收数量和 receipt 明细一致
已入库收货生成库存流水或被标记为历史已入库
未入库收货不直接增加可用库存

7.9 历史出货

注意

历史出货会影响：

销售订单已发数量
库存扣减
应收事实
客户交付状态

必须确认历史出货是否已经体现在期初库存中。

关键选择

如果期初库存已经是当前库存：

历史出货不应该再次扣库存
只用于历史查看和销售订单出货进度

如果导入从历史库存流水重放：

历史出货可以生成 SHIPMENT_OUT 流水

测试

历史出货不会重复扣当前库存
销售订单 shipped_qty 正确
重复导入同一出货单不会重复生成
应收事实不重复

7.10 财务期初和业务财务事实

注意

当前系统是业务财务事实，不是正式总账。

导入时必须区分：

期初应收
期初应付
历史已收
历史已付
未结余额

测试

应收金额 $\geq$ 已收金额
应付金额 $\geq$ 已付金额
未收金额计算正确
未付金额计算正确
已结清状态 PAID
未结清状态 OPEN 或 PARTIALLY_PAID
不会生成总账凭证

8. 未匹配队列规范

8.1 什么进入 unresolved queue

以下情况不能自动导入：

未知客户
未知供应商
未知 SKU
未知物料
未知单位
未知仓库
BOM 物料无法匹配
订单状态无法映射
数量非法
金额非法
日期无法解析
重复单号冲突
hash 不匹配

8.2 unresolved item 字段

```
{
  "id": "unresolved-001",
  "importJobId": "import-yoyoosun-001",
  "sourceId": "bom-26029-v1",
  "rowNumber": 23,
  "objectType": "material",
  "rawValue": "短毛绒粉色",
  "reason": "material_not_matched",
  "suggestions": [
    {
      "targetId": "mat-001",
      "targetName": "粉色短毛绒",
      "confidence": 0.82
    }
  ],
  "status": "OPEN"
}
```

8.3 处理方式

匹配到已有数据
新建主数据
忽略
标记阻塞
批量规则处理

8.4 测试

unresolved queue 可生成
处理后 dry-run 可继续
忽略项进入报告
阻塞项不允许 apply
所有处理动作写 audit event

9. 导入 Dry-run 规范

9.1 Dry-run 必须检查

文件存在性
文件 hash
字段完整性
数据类型
日期格式
金额格式
数量格式
单位映射
主数据映射
重复数据
状态映射
业务规则
外键引用
是否会覆盖已有数据
是否会产生库存变化
是否会产生财务事实

9.2 Dry-run 输出

```
{
  "importJobId": "import-yoyoosun-001",
  "customerCode": "yoyoosun",
  "status": "failed",
  "summary": {
    "sourceCount": 12,
    "rowCount": 5320,
    "validRows": 5100,
    "warningRows": 180,
    "errorRows": 40,
    "unresolvedCount": 35
  },
}
```

```
"checks": [  
  {  
    "name": "source_manifest",  
    "status": "passed"  
  },  
  {  
    "name": "material_mapping",  
    "status": "failed",  
    "errorCount": 20  
  }  
],  
"blockingErrors": [  
  {  
    "sourceId": "bom-26029-v1",  
    "rowNumber": 23,  
    "message": "material not matched"  
  }  
]  
}
```

9.3 Dry-run 通过条件

必须满足：

```
required source 全部存在  
hash 全部匹配  
blocking errors = 0  
unresolved blocking = 0  
重复策略明确  
导入影响范围明确  
客户已确认
```

10. Apply 导入规范

10.1 Apply 前必须做

```
确认 dry-run report  
确认 unresolved queue 已处理  
确认客户签收  
确认数据库备份完成  
确认当前 migration status 正确  
确认 APP_ENV 正确  
确认 production preflight 通过
```

确认 release version 固定
确认导入脚本版本固定

10.2 Apply 必须有批次

每次 apply 必须生成：

import_batch_id

每条导入数据必须记录：

import_batch_id
source_id
source_row
source_hash
created_by_import

10.3 Apply 必须幂等

重复执行同一批次时：

不能重复创建客户
不能重复创建 SKU
不能重复创建 BOM
不能重复增加库存
不能重复创建销售订单
不能重复扣库存
不能重复生成财务事实

10.4 Apply 后必须验证

主数据数量
订单数量
BOM 数量
库存余额
库存流水
财务事实
unresolved queue

11. 每个甲方部署环境规范

11.1 环境类型

环境	用途	数据
local	开发	fake/test
dev	内部开发	fake/sanitized
staging	客户试用预演	sanitized 或客户授权数据
customer-trial	客户试用	客户确认数据
production	正式生产	客户正式数据

11.2 不同环境允许的数据

数据类型	local	dev	staging	customer-trial	production
test fixture	是	是	否	否	否
demo seed	是	是	可选	否	否
脱敏客户样例	是	是	是	是	否
客户真实数据	否	否	需授权	是	是
mock SMS	是	是	否	否	否
debug cleanup	是	可选	否	否	否
SQL args tracing	可选	可选	否	否	否

11.3 部署配置注入

禁止把真实配置写入 git。

推荐：

```
.env.example 入库
.env 不入库
CI Secret
服务器环境变量
```

Docker Secret
Kubernetes Secret
私有化部署配置文件受控保存

12. 客户部署前检查清单

12.1 配置检查

- [] customerCode 正确
- [] APP_ENV 正确
- [] timezone 正确
- [] defaultCurrency 正确
- [] 菜单配置正确
- [] 权限配置正确
- [] 编号规则正确
- [] feature flags 正确
- [] public register 关闭
- [] sms mock 关闭
- [] debug seed 关闭
- [] debug cleanup 关闭
- [] SQL args tracing 关闭
- [] CORS 非 *
- [] JWT secret 非 placeholder
- [] DB password 非 placeholder
- [] admin password 非弱密码

12.2 Seed 检查

- [] seed-manifest 存在
- [] required seed 文件存在
- [] seed hash 匹配
- [] seed dry-run 通过
- [] seed 幂等测试通过
- [] 无 demo seed
- [] 无 test fixture
- [] super admin bootstrap 只执行一次
- [] 角色权限正确
- [] 默认仓库/单位正确

12.3 导入检查

```
[ ] source-manifest 存在
[ ] required source 文件存在
[ ] source hash 匹配
[ ] mapping 文件存在
[ ] dry-run report 通过
[ ] unresolved queue 已处理
[ ] blocking errors = 0
[ ] 客户确认 dry-run 结果
[ ] 导入前数据库备份完成
[ ] apply 脚本版本固定
```

12.4 部署检查

```
[ ] release package 生成
[ ] release package 无 secret
[ ] docker image 版本固定
[ ] migration status 正确
[ ] backup restore 演练通过
[ ] smoke test 通过
[ ] known limitations 已提供
[ ] acceptance checklist 已提供
[ ] rollback plan 已提供
```

13. 自动化测试方案

13.1 客户配置测试

测试文件建议

```
scripts/customer/config-lint.test.mjs
```

测试内容

```
customer.config.json schema 校验
customerCode 合法
timezone 合法
currency 合法
menu config key 合法
field alias 不冲突
numbering rule 可生成唯一编号
```

feature flags 不包含生产危险开关
permissions 中权限 key 合法

验收

```
node --test scripts/customer/config-lint.test.mjs -- --customer yoyoosun
```

13.2 Seed 测试

测试文件建议

```
scripts/customer/seed.test.mjs  
server/test/integration/customer_seed_test.go
```

测试内容

```
seed-manifest schema 校验  
required seed 文件存在  
hash 匹配  
seed dry-run  
seed apply  
重复 seed apply  
created/updated/skipped 数量合理  
无 demo/test 数据  
bootstrap admin 只执行一次
```

核心断言

```
第一次执行 created > 0  
第二次执行 duplicated = 0  
第二次执行 skipped >= 第一次 created  
super_admin 不重复创建  
库存期初不重复增加
```

13.3 导入测试

测试文件建议

```
scripts/customer/import-dry-run.test.mjs  
scripts/customer/import-apply.test.mjs  
server/test/integration/customer_import_test.go
```

Dry-run 测试

```
manifest schema  
source hash  
mapping schema  
字段缺失  
未知单位  
未知物料  
重复 SKU  
BOM 物料无法匹配  
期初库存负数  
历史订单状态映射
```

Apply 测试

```
apply 前必须备份  
apply 必须有 import_batch_id  
apply 后数量正确  
重复 apply 不重复创建  
库存余额正确  
财务事实正确  
审计日志存在
```

13.4 部署测试

测试文件建议

```
scripts/customer/deploy-smoke.test.mjs  
scripts/customer/customer-trial-package.test.mjs  
scripts/deploy/verify-customer-deployment.sh
```

测试内容

```
release package 文件完整  
.env 不在包内
```



```
.env.example 在包内  
migration 可执行  
preflight 通过  
seed dry-run 通过  
seed apply 通过  
import dry-run 通过  
smoke test 通过  
backup restore 通过
```

13.5 Smoke Test

客户部署后必须执行：

```
登录 admin  
查看首页  
查看客户列表  
查看 SKU 列表  
查看 BOM 列表  
查看销售订单列表  
查看采购订单列表  
查看库存余额  
查看库存流水  
查看审计日志  
创建一条测试草稿订单  
删除或作废测试草稿订单  
确认没有 demo 数据
```

注意：

生产 smoke 不应该创建正式业务事实。
如需测试写入，只能创建标记为 SMOKE_TEST 的草稿，并在测试结束清理或作废。

14. 部署流程

14.1 部署前

1. 确认 release version
2. 确认 customer config version
3. 确认 seed manifest version
4. 确认 source manifest version
5. 运行 config lint
6. 运行 seed dry-run

7. 运行 import dry-run
8. 生成 dry-run reports
9. 客户确认 reports
10. 做数据库备份
11. 确认 rollback plan

14.2 部署中

1. 拉取固定版本镜像
2. 注入环境变量
3. 执行 production preflight
4. 执行数据库 migration
5. 执行 system seed
6. 执行 customer seed
7. 执行 import apply
8. 执行 post-apply verification
9. 执行 smoke test
10. 生成 deployment evidence

14.3 部署后

1. 检查系统日志
2. 检查审计日志
3. 检查导入报告
4. 检查关键数据数量
5. 检查首页 KPI
6. 检查权限菜单
7. 检查备份任务
8. 客户签收
9. 归档 evidence

15. 回滚方案

15.1 必须有两类回滚

应用版本回滚
数据导入回滚

15.2 应用版本回滚

停止新版本
恢复旧镜像
确认配置兼容
确认 migration 是否可逆
执行 smoke test

15.3 数据导入回滚

优先级：

1. 使用数据库备份整体恢复
2. 使用 import_batch_id 定位并执行反向操作
3. 对库存/财务只能用反向事实，不直接删除事实

注意

库存和财务不建议硬删除。

错误导入应通过：

反向库存流水
取消财务事实
归档错误业务单据
重新导入正确批次

16. 每个甲方交付报告

每次客户试用或生产部署必须生成：

customer-delivery-report.md

建议结构：

```
# 客户交付报告

## 基本信息
- customerCode:
- releaseVersion:
```

```
- configVersion:
- seedVersion:
- importBatch:
- deployTime:
- operator:

## 配置结果
- config lint:
- feature flags:
- menu:
- permissions:

## Seed 结果
- seed status:
- created:
- updated:
- skipped:
- failed:

## 导入结果
- source count:
- row count:
- created:
- updated:
- skipped:
- failed:
- unresolved:
- blocking errors:

## 验证结果
- smoke test:
- migration status:
- backup restore:
- security scan:
- preflight:

## 已知限制
- limitation 1:
- limitation 2:

## 回滚方案
- backup:
- rollback command:
- rollback owner:

## 客户签收
- signer:
- signedAt:
```

17. Codex 任务拆分

CUST-01：客户配置包结构

请实现 CUST-01：客户配置包结构。

要求：

1. 新增 customers/<customer-code>/ 标准目录模板。
2. 增加 config、seed、import、deployment、reports、evidence 子目录。
3. 增加 README.md，说明每个目录用途。
4. 不要提交真实客户 secret 或 raw files。
5. 增加 docs/customer-delivery/ 说明文档。

CUST-02：客户配置 Schema 和 Lint

请实现 CUST-02：客户配置 Schema 和 Lint。

要求：

1. 定义 customer.config.json、menu.config.json、field-alias.config.json、numbering.config.json、feature-flags.config.json、permissions.config.json 的 JSON schema。
2. 新增 scripts/customer/config-lint.mjs。
3. 新增 scripts/customer/config-lint.test.mjs。
4. 检查 production 禁止 publicRegister、smsMock、debugSeed、debugCleanup、sqlArgsTracing。
5. 检查 menu key、permission key 合法。
6. 运行 node --test scripts/customer/*.test.mjs。

CUST-03：Seed Manifest 和 Seed Dry-run

请实现 CUST-03：Seed Manifest 和 Seed Dry-run。

要求：

1. 定义 seed-manifest.json schema。
2. 新增 scripts/customer/seed-dry-run.mjs。
3. 检查 required seed 文件存在、hash 匹配、执行顺序、重复 seedId。
4. 检查 seed 中不能包含 demo/test 数据。
5. 输出 seed-dry-run-report.json。
6. 增加测试。

CUST-04 : Seed Apply 幂等

请实现 CUST-04 : Seed Apply 幂等。

要求：

1. 实现 customer seed apply。
2. 每条 seed 使用稳定 key, 重复执行不重复创建。
3. bootstrap admin 只允许一次。
4. 禁止生产执行 demo seed。
5. 输出 seed-apply-report.json。
6. 增加 Go 或 Node 测试, 验证重复 apply 不产生重复数据。

CUST-05 : Source Manifest 和 Import Mapping

请实现 CUST-05 : Source Manifest 和 Import Mapping。

要求：

1. 定义 source-manifest.json schema。
2. 定义 mapping schema。
3. 新增 scripts/customer/import-dry-run.mjs。
4. 检查 sourceId 唯一、required 文件存在、sha256 匹配、mapping 字段合法。
5. 不依赖中文 originalName。
6. 输出 import-dry-run-report.json。
7. 增加测试。

CUST-06 : Unresolved Queue

请实现 CUST-06 : Unresolved Queue。

要求：

1. 设计 unresolved queue 数据结构。
2. 导入 dry-run 遇到未知客户、供应商、SKU、物料、单位、仓库时写 unresolved queue。
3. unresolved blocking 未处理时禁止 apply。
4. 支持 match existing、create new、ignore、block。
5. 所有处理动作写 audit event。
6. 增加测试。

CUST-07 : Import Apply 和批次幂等

请实现 CUST-07 : Import Apply 和批次幂等。

要求：

1. apply 必须依赖通过的 dry-run report。
2. 每次 apply 生成 import_batch_id。
3. 每条导入数据记录 source_id、source_row、source_hash、import_batch_id。
4. 重复执行同一批次不重复创建、不重复加库存、不重复生成财务事实。
5. 输出 import-apply-report.json。
6. 增加测试。

CUST-08 : 客户部署 Smoke Test

请实现 CUST-08 : 客户部署 Smoke Test。

要求：

1. 新增 scripts/customer/deploy-smoke.mjs。
2. 支持传入 customerCode 和 endpoint。
3. 验证登录、首页、客户列表、SKU 列表、BOM 列表、销售订单列表、采购订单列表、库存余额、库存流水、审计日志。
4. 生产环境不要创建正式业务事实。
5. 输出 smoke-test-report.json。
6. 增加测试。

CUST-09 : 客户试用包生成

请实现 CUST-09 : 客户试用包生成。

要求：

1. 新增 scripts/customer/build-customer-package.mjs。
2. 生成 release/customers/<customer-code>/ 目录。
3. 包含 config、seed manifest、source manifest、mapping、reports、deployment checklist、rollback plan、known limitations。
4. 不包含真实 .env、真实 token、raw customer files。
5. 构建前跑 config lint、seed dry-run、import dry-run、secret scan。
6. 增加测试。

CUST-10：客户交付报告

请实现 CUST-10：客户交付报告。

要求：

1. 新增 scripts/customer/generate-delivery-report.mjs。
2. 聚合 config lint、seed report、import report、smoke test、security scan、backup restore、known limitations。
3. 输出 customer-delivery-report.md。
4. 增加测试。

18. 人工 Review 清单

每个客户配置/导入/部署 PR 必须检查：

- [] 是否包含真实 secret
- [] 是否包含真实客户 raw files
- [] customerCode 是否稳定
- [] config 是否通过 schema 校验
- [] feature flags 是否安全
- [] menu 是否符合客户试用范围
- [] permissions 是否后端可执行
- [] seed 是否幂等
- [] 是否混入 demo/test 数据
- [] source manifest 是否完整
- [] hash 是否记录
- [] mapping 是否明确
- [] dry-run 是否通过
- [] unresolved queue 是否处理
- [] apply 前是否备份
- [] apply 是否有 import_batch_id
- [] 期初库存是否通过流水写入
- [] 历史出货是否避免重复扣库存
- [] 财务事实是否不是总账凭证
- [] smoke test 是否通过
- [] rollback plan 是否可执行
- [] customer delivery report 是否生成

19. 最短落地顺序

如果只想先做最关键的客户交付能力，按这个顺序：

1. CUST-01 客户配置包结构
2. CUST-02 客户配置 Schema 和 Lint
3. CUST-03 Seed Manifest 和 Seed Dry-run
4. CUST-04 Seed Apply 幂等
5. CUST-05 Source Manifest 和 Import Mapping
6. CUST-06 Unresolved Queue
7. CUST-07 Import Apply 和批次幂等
8. CUST-08 客户部署 Smoke Test
9. CUST-09 客户试用包生成
10. CUST-10 客户交付报告

完成这 10 项后，每个甲方的配置、seed、导入、测试和部署就会有稳定闭环。

20. 最终原则

一句话：

每个甲方都必须有独立、可验证、可回滚的交付包。

更具体地说：

配置要可 lint。
seed 要幂等。
导入要 dry-run。
apply 要有批次。
库存要有流水。
财务要有边界。
部署要有 preflight。
上线要有 smoke。
失败要能回滚。
交付要有报告。